

QA PLATFORM • PROJECT PRESENTATION

Manual Testing & Automated Testing, End to End

How both testing modes work inside a full-stack QA management platform — the workflows, the real code, and the technology behind them.

React 19 / Vite • Flask 3 / MySQL • Playwright / axe-core • Gemini AI

Backend suite: 312 pytest tests passing • Frontend: 14 Vitest tests, ESLint clean, production build green

What this deck covers

- **01 The problem** — why a project needs both manual and automated testing
- **02 Project overview** — what QA Platform is and how it is built
- **03 Manual testing** — the human-driven workflow, step by step
- **04 Automated testing** — the Playwright crawl engine, and exactly how it detects issues
- **05 What each mode detects** — the 8 categories both share, and what only one can see
- **06 Technology deep dive** — every tool, what it does, and the fact that no API key is needed
- **07 Quality of the platform** — our own test automation and results
- **08 Engineering challenges** — real bugs we found and how we fixed them
- **09 Limitations & roadmap** — what is not solved yet
- **10 Key learnings** — transferable practices

01

The Problem

Manual testing and automated testing are not competitors. They fail in opposite directions, which is exactly why a real QA platform needs both.

- Manual testing: high judgement, terrible coverage, does not scale
- Automated testing: perfect repeatability, zero judgement
- Most teams pick one and pay for it in escaped bugs
- QA Platform runs both, and keeps their results strictly separate

Why one testing mode is never enough

Manual only

- A tester eyeballs maybe 5–10 screens per hour
- Nobody re-runs the same 60 checks after every hotfix
- Exploratory bugs hide in the gaps between scripted paths
- One person off sick means zero coverage
- Consistency depends entirely on who is holding the mouse

Automation only

- A crawler cannot tell if a button *feels* broken
- Scripts encode yesterday's understanding of the product
- Visual and UX regressions slip past DOM assertions
- A green build gets reported as "quality is fine"
- Nobody remembers the checkout flow was rewritten

DESIGN PRINCIPLE

The gap this project fills: run human judgement and machine coverage *in the same platform*, on the same test cases, and report both — without either one overwriting the other.

02

Project Overview

A full-stack application for managing the entire software quality workflow — test cases, executions, evidence, defects, reports and AI.

- Projects, test cases, runs, bugs, reports, admin
- React 19 + Vite frontend, Flask REST API, MySQL persistence
- Playwright drives a real Chromium for every automated run
- JWT auth, role-based admin, AI assistance via Google Gemini

QA Platform at a glance

01

Test case management

Manual and automated test cases with steps, expected results, priority and status. AI can propose the cases for you.

02

Manual execution

A real browser window opens, the tester logs in by hand and works through a checklist. Every step is screenshotted.

03

Automated execution

A Playwright crawler audits up to 100 pages for links, console errors, accessibility, security, performance and more.

04

Evidence capture

Full-page screenshots plus pixel-cropped images around each finding at its exact bounding box.

05

Bug tracking

Findings convert into tracked bugs in one click, with reproduction steps written per defect category.

06

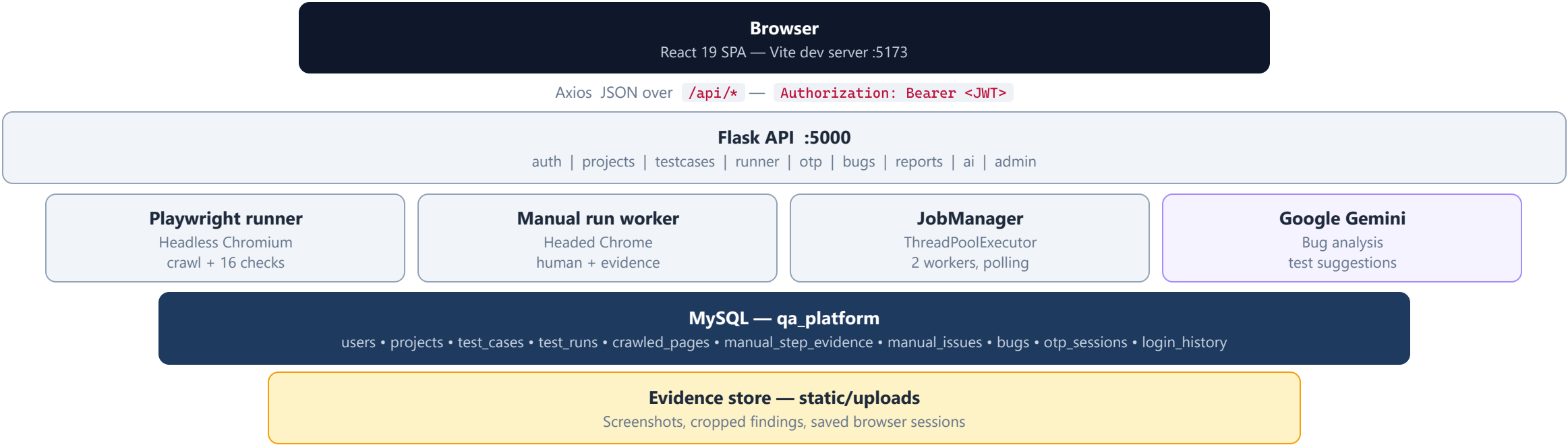
Reports & AI

Health trends, pass rates and PDF export, plus Gemini-powered bug analysis and test-case suggestions.

The technology behind the platform

Layer	Technology	Version	Role in testing
Frontend	React + Vite	19.2 / 8.0	Test-case UI, run modal, live progress, reports
Styling / UI	Tailwind CSS, Framer Motion, Lucide	3.4 / 12.4	Interface, modals, checklist interactions
Charts / 3D	Recharts, Three.js, drei	3.8 / 0.184	Health trends, pass-rate graphs
HTTP	Axios	1.16	Typed API client with JWT injection
Backend API	Python + Flask	3.1.3	9 blueprints, all under <code>/api</code>
Auth	Flask-JWT-Extended + bcrypt	4.7.3 / 5.0	JWT access tokens, password hashing, role checks
Database	MySQL via Flask-MySQLdb	8.x / 2.0	9 tables, DictCursor, utf8mb4
Browser engine	Playwright	1.59.0	Drives real Chromium for all automated runs
Accessibility	axe-core (vendored)	4.10.2	WCAG 2.1 A/AA audit injected into each page
Imaging	Pillow	12.2.0	Crops screenshots around each finding
Reports	ReportLab	4.5.0	Server-side PDF generation
AI	Google Gemini (REST)	flash	Bug analysis, test suggestions, chat assistant

System architecture



03

Manual Testing

A human tester drives a real, visible browser. The platform supplies the checklist, captures the evidence, and quietly automates everything around the tester.

- Headed Chrome opened by the backend, not a new tab
- Written steps become a live pass / fail checklist
- Every navigation screenshotted automatically
- Automatic checks run alongside — but never decide the verdict

How manual testing works in QA Platform

1

Create a **manual** test case: title, steps, expected result

2

Click **Run Manual Test** → backend opens a headed Chrome

3

Tester logs in by hand (SSO, OTP, captcha — all fine)

4

Tester walks the checklist, ticking pass / fail / skip per step

5

Platform screenshots every page and runs automatic checks

6

Optional: report a visual issue with expected vs actual

7

Answer the expected-result question: **Met** or **Not met**

8

Submit → run row, step evidence and issues written to MySQL

DIVISION OF LABOUR

The tester supplies **judgement**. The platform supplies **consistency** — the checklist, the evidence trail, the storage, and the checks a human would forget on page 12.

Step 1–2: the test case contract and the browser launch

Manual cases must declare steps + expected result; crawl_pages is derived server-side, never trusted from the client.

```
# backend/modules/testcases/routes.py : 120
steps = (data.get('steps') or '').strip()
if test_type == 'manual' and not steps:
    return jsonify({'error': 'Steps are required for manual tests'}), 400

expected_result = (data.get('expected_result') or '').strip()
if test_type == 'manual' and not expected_result:
    return jsonify({'error': 'Expected result is required'}), 400

# Automated runs crawl the site by contract. The UI never exposes
# crawling as a separate yes/no -- an automated test always crawls
# up to max_pages -- so trusting the client here would let a request
# ask for an automated test that silently audits a single page.
crawl_pages = (test_type == 'automated')
```

Headed real Chrome, driven by the backend on its own thread.

```
# backend/modules/runner/manual_run.py : 347
with sync_playwright() as p:
    # Prefer the user's real installed Chrome so the fingerprint is
    # indistinguishable from a normal browser; fall back to bundled
    # Chromium if Chrome isn't installed.
    stealth_args = ['--start-maximized',
                    '--disable-blink-features=AutomationControlled']
    browser = p.chromium.launch(
        channel='chrome', headless=False, args=stealth_args,
    )
    context = browser.new_context(no_viewport=True)
    context.add_init_script(
        "Object.defineProperty(navigator, 'webdriver', "
        "{get: () => undefined});"
    )
    page = context.new_page()
```

The auto-crawl trigger: never crawl before the tester has logged in

This function was rewritten to fix a real bug we hit in testing. See the Challenges section.

```
# backend/modules/runner/manual_run.py : 158
def should_auto_trigger(current_url, entry_url, stable_secs,
                        navigated, threshold=None):
    """Decide whether to start the automatic crawl.

    The crawl fires only once the tester has actually gone somewhere.
    The old rule -- "off the login page and stable for 5s" -- also
    fired when the run *started* on a page that merely looks public:
    a login form served at '/' or a custom auth path the login
    detector does not recognise. That crawled before the tester had
    logged in. Sitting still on the entry page is never evidence of
    being logged in; navigating is.
    """
    if threshold is None:
        threshold = AUTO_TRIGGER_STABLE_SECS
    if _looks_like_login(current_url):
        return False
    if stable_secs < threshold:
        return False
    if not navigated and _same_url(current_url, entry_url):
        return False
    return True
```

orange

The bug

A run started on a page that *looked* public — a login form served at `/` — passed the detector, so the crawler started before the tester had authenticated.

ok

The fix

Navigation is now required, not just a non-login-looking URL. Sitting still never triggers; only actually going somewhere does.

blue

The crawl itself

Phase 2 is a BFS crawl using **in-app link clicks** rather than `page.goto`, so React stays mounted and the auth session survives the whole crawl.

The verdict contract: human judgement is never overwritten

The UI labels the human column “HUMAN OBSERVATION (decides the verdict)” to make the precedence visible to the tester.

```
# backend/modules/runner/routes.py : 2477
issues_found = len(functional) + len(reported) + \
    len(snap.get('auto_findings') or [])

# Manual verdict depends ONLY on the tester's steps, expected-result
# decision and reported issues. Automated evidence is stored alongside
# but never overrides it (see manual_run.summarise_manual).
if snap.get('expected_met') is False:
    verdict_reason = 'Tester marked the expected result as not met.'
elif any(s['status'] == 'failed'
        for s in (snap.get('steps') or [])):
    n = sum(1 for s in (snap.get('steps') or [])
           if s['status'] == 'failed')
    verdict_reason = f'{n} manual step(s) failed.'
elif reported:
    verdict_reason = f'{len(reported)} issue(s) reported by the tester.'

# Persisted with an explicit provenance stamp:
#   run_type      = 'MANUAL'
#   execution_mode = 'HEADED'
#   source        = 'HUMAN' (or 'HYBRID' if auto-findings exist)
```

TRUST

Why this matters: if automatic checks could flip a manual verdict, testers would stop trusting the report. A tool that overrides the human is a tool the team routes around.

What the platform automates *around* the human

Automated by the platform	Judged by the tester
Opens and manages the real Chrome window	Is the flow usable and understandable?
Turns the written steps into a live checklist	Does each step actually do what it claims?
Screenshots every page navigation, automatically	Does the page look and feel correct?
Runs alt-text, SEO, axe-core a11y, console, URL, UI and code checks	Is the content right, the copy correct, the tone appropriate?
Crawls deeper pages with BFS once login is done	Is anything confusing, misleading or unexpectedly missing?
Captures console output and timings as supporting evidence	Is this a defect worth tracking, and how severe is it?
Stores steps, notes, screenshots and issues in MySQL	Met or not met — the one input that sets Pass / Fail

A DELIBERATE OMISSION

Security-header and mobile checks are deliberately **skipped** in manual mode. A headed desktop window would report every one of them falsely — a desktop viewport cannot fail a 44 px tap-target test. Running them would be noise, and noise trains people to ignore the report.

Keeping manual and automated results apart — four layers

1

Entry guard

The synchronous automated endpoint rejects any case whose `test_type ≠ 'automated'`, and the manual start endpoint rejects unknown cases. Neither can be tricked by the other.

2

Type enum

A validator accepts only `manual` or `automated`. Manual runs have no automation framework field to fill in at all.

3

Storage

Manual evidence lands in dedicated tables — `manual_step_evidence` and `manual_issues` — and every finding is stamped `source='MANUAL'`. Automated regression logic explicitly refuses to compare against it.

4

Reporting

Every finding renders with a MANUAL / AUTOMATED badge, and the reports API filters on `run_type` and `source`.

A dedicated test file, `tests/test_run_separation.py`, asserts this contract so a future refactor cannot quietly merge the two streams.

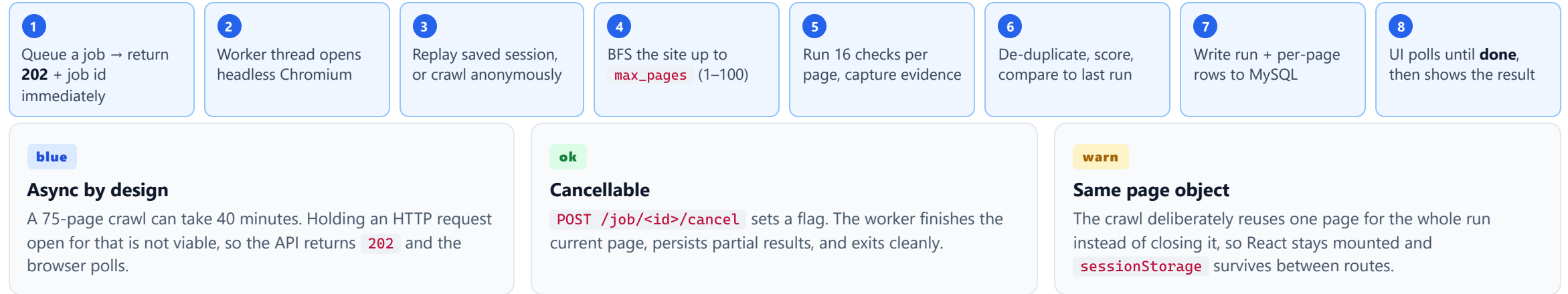
04

Automated Testing

A Playwright crawler drives headless Chromium through a site and applies 16 independent checks per page, then scores and persists the result.

- Asynchronous jobs, polled live by the React UI
- Authenticated crawling by replaying a saved browser session
- axe-core WCAG 2.1 A/AA audit injected into every page
- De-duplicated findings and a weighted health score

The engine pipeline



Asynchronous execution: submit, then poll

Ownership, type and SSRF checks all happen before a job is ever queued.

```
# backend/modules/runner/routes.py : 2201
@runner_bp.route('/run/<int:testcase_id>/async',
                 methods=['POST'])
@jwt_required()
def run_test_async(testcase_id):
    """Queue the run on a background worker and return a job id."""
    user_id = int(get_jwt_identity())
    tc = user_owns_testcase(user_id, testcase_id)
    if not tc:
        return jsonify({'error': 'Test case not found'}), 404
    if tc['test_type'] != 'automated':
        return jsonify({'error':
            'Only automated test cases can be run'}), 400
    ok, reason = _is_allowed_test_url(tc.get('base_url'))
    if not ok:
        return jsonify({'error': reason,
            'code': 'invalid_url'}), 400

    job_id = job_manager.submit(user_id, testcase_id, dict(tc))
    return jsonify({'job_id': job_id, 'status': 'queued'}), 202
```

Live progress every 2 seconds; NaN-safe timing.

```
# frontend/src/pages/TestCases.jsx : 731
const pollRunJob = (tcId, jobId) =>
  new Promise((resolve, reject) => {
    const tick = async () => {
      const res = await getRunJob(jobId);
      const job = res.data;
      if (job.progress) setRunProgress(tcId, job.progress);
      if (job.status === 'done' || job.status === 'failed' ||
          job.status === 'cancelled') {
        resolve(job);
      } else {
        setTimeout(tick, 2000);
      }
    }
  })
```

HONEST ERROR HANDLING

A `404` mid-poll now reports “the run was interrupted — the backend restarted” instead of “Job not found”. The in-memory job registry is a known limitation; a `run_jobs` table is the planned fix.

The background worker

Two workers, a lock around the registry, and LRU eviction that never drops a live job.

```
# backend/modules/runner/jobs.py : 32
class JobManager:
    """Thread-backed job registry. In-memory: fine for a single
    process. For multiple processes move job state to Redis/DB."""
    def init(self, app, max_workers=2):
        self._executor = ThreadPoolExecutor(max_workers,
                                           thread_name_prefix='runner')

    def submit(self, user_id, testcase_id, tc):
        job_id = uuid4().hex
        with self._lock:
            self._jobs[job_id] = {
                'job_id': job_id, 'status': 'queued',
                'cancel_requested': False, ...}
        try:
            self._executor.submit(self._run, job_id, testcase_id, tc)
        except RuntimeError: # pool already shut down
            self._jobs.pop(job_id) # never leave a job stuck "queued"
        return job_id

    def _run(self, job_id, testcase_id, tc):
        # Each worker thread needs its own Flask app context, else
        # mysql.connection cannot bind on that thread.
        with self._app.app_context():
            result, code = _perform_run(testcase_id, tc,
                                       progress_cb=_progress, # → job['progress']
                                       cancelled_check=_cancelled) # → cooperative cancel
```

blue

State machine

queued → running → done | failed | cancelled .
Cancellation wins over a success return code.

warn

Crash handling

Any worker exception becomes `status='failed'` with
`error_code='worker_crash'` .

ok

Eviction

Only *finished* jobs are evicted, oldest first, at 200 retained. A long crawl is never dropped.

Browser context, session replay and anti-detection

The viewport choice is not cosmetic: it decides whether the mobile tap-target checks are meaningful or pure noise.

```
# backend/modules/runner/routes.py : 2752
with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)

    # When we have a captured session, use a DESKTOP context that
    # matches the session_capture window. Many sites bind auth
    # cookies to the originating UA -- using a mobile UA
    # invalidates the session. Without a session we keep the mobile
    # viewport so the mobile-issues checks still mean something.
    if session_used:
        context_kwargs = {
            'viewport': {'width': 1366, 'height': 768},
            'user_agent': ('Mozilla/5.0 (Windows NT 10.0; Win64; x64) '
                          '... Chrome/131.0.0.0 Safari/537.36'),
            'storage_state': session_path,
        }
    else:
        context_kwargs = {
            'viewport': {'width': 390, 'height': 844}, # iPhone
            'user_agent': 'Mozilla/5.0 (iPhone; CPU iPhone OS 16_0 '
                          'like Mac OS X)',
        }
    context = browser.new_context(**context_kwargs)
    context.add_init_script(
        "Object.defineProperty(navigator, 'webdriver', "
        "{get: () => undefined});")
```

DESIGN NOTE

Why 1366×768 for authenticated runs? Mobile checks (44 px tap targets, horizontal scroll) are meaningless in a desktop viewport, so they are skipped. An earlier version ran them anyway and produced a page of false positives on every authenticated crawl.

The check matrix — 16 checks per page

#	Check function	Detects	Severity ceiling
1	<code>check_broken_links</code>	Dead links, tested with the logged-in user's cookies and real User-Agent, 8-way parallel, HEAD→GET fallback	serious
2	<code>check_missing_alt</code>	Images with no alt text (skips decorative, aria-hidden, <32 px)	minor
3	<code>check_seo</code>	Missing title, missing/empty meta description, missing h1	moderate
4	<code>check_security</code>	Non-HTTPS, missing CSP / X-Frame-Options / HSTS	critical
5	<code>check_accessibility_axe</code>	axe-core WCAG 2.1 A/AA violations, with a heuristic fallback	from axe impact
6	<code>check_mobile</code>	Horizontal scroll, tap targets under 44×44 px	serious
7	<code>check_dead_controls</code>	Buttons and links that do nothing (opt-in — can submit forms or log you out)	serious
8	<code>check_performance</code>	Slow pages, heavy payloads, heaviest resources	serious
9	<code>filter_console_errors</code>	Real console errors, with extension/deprecation noise filtered out	serious
10	<code>check_api_responses</code>	4xx / 5xx API responses with human-readable status meanings	critical
11	<code>check_form_validation</code>	Required fields with no <code>checkValidity()</code> enforcement	serious
12	<code>check_url_health</code>	Off-site redirects, session IDs in query strings, malformed URLs	serious
13	<code>check_api_quality</code>	Slow APIs, mixed content, APIs returning HTML, repeated failures	serious
14	<code>check_page_resources</code>	Empty src slots, dead <code>#fragment</code> anchors	moderate
15	<code>check_ui_consistency</code>	Duplicate element IDs, empty buttons, missing viewport meta, overflow	serious
16	<code>check_code_quality</code>	Inline <code>onclick</code> , <code>eval()</code> , deprecated tags, jQuery < 3	serious

How a page is actually audited: the fan-out

One loaded page in, 15 categories of findings out. Every check has the same contract: (page, ...) -> list[finding].

```
# backend/modules/runner/routes.py : 1850
def test_single_page(page, page_url, response_headers,
                    console_messages, page_load_ms=0,
                    api_responses=None, mobile_checked=None,
                    page_status=0):
    """Run every check against one loaded page.

    Returns (findings_by_category, coverage). `coverage` records
    what was capped or skipped so the report can say so out loud
    instead of letting a partial check look like a clean bill of
    health."""
    if mobile_checked is None:
        mobile_checked = bool(is_mobile)
    broken_links, link_coverage = check_broken_links(page, page_url)
    missing_alt = check_missing_alt(page)
    seo_issues = check_seo(page, page_url)
    security_issues = check_security(page_url, response_headers)
    accessibility_issues, ally_engine = check_accessibility(page)

    if mobile_checked:
        mobile_issues, mobile_coverage = check_mobile(page)
        mobile_skipped_reason = None
    else:
        mobile_issues, mobile_coverage = [], {}
        mobile_skipped_reason = ('disabled: page was audited '
                                'in a desktop viewport')
```

1 JS executed inside the page

Most checks call `page.evaluate("() => ...")`. Example — `check_form_validation` blanks every `required` control, calls `checkValidity()`, then reports a form that declares required fields yet accepts an empty submission.

2 A third-party engine

axe-core is injected with `add_script_tag` and runs `axe.run()` for the WCAG tags. Its violations come back as findings, tagged with the runner's own severities.

3 Out-of-band observation

Console errors, network responses, HTTP status and timing **cannot be read from the DOM at all**. The crawler attaches `page.on('console')`, `on('request')` and `on('response')` once, and those collected messages are *passed into* this function.

The crawler is a **passive listener first and an active prober second** — it watches everything that happens while the page loads, then asks questions. The one check that can say “the feature is broken” is the step executor: a failed step becomes a `functional_issues` finding rated **critical**.

The 8 issue categories detected in *both* modes

One detection library, two policies. `_auto_check()` in `manual_run.py:593` imports the *same functions* the crawler calls, so these eight categories are found in a manual run and an automated run alike.

Category	What it actually finds	Severity
Missing alt images	<code></code> with no <code>alt</code> — excluding decorative (<code>alt=""</code>), <code>role=presentation</code> , <code>aria-hidden</code> , and images under 32 px	minor
SEO	Missing <code><title></code> , missing or empty <code>meta[name=description]</code> , missing <code><h1></code>	moderate
Accessibility	axe-core 4.10.2 WCAG 2.1 A/AA violations (<code>wcag2a</code> , <code>wcag2aa</code> , <code>wcag21a</code> , <code>wcag21aa</code>), violations only. <code>image-alt</code> excluded — owned by missing-alt	axe's own impact
Console errors	Real JavaScript errors, with extension and deprecation noise filtered out. Mixed content is deliberately <i>not</i> skipped	serious
URL health	Off-site 301/302/307/308 redirects, session IDs in the query string (<code>jsessionid</code> , <code>phpsessid</code> , <code>sid</code>), unencoded spaces, URLs over 2000 chars, uppercase path	serious–minor
UI consistency	Duplicate element IDs, empty buttons and links, missing <code><meta viewport></code> , missing <code>html lang</code> , horizontal overflow	serious–minor
Code quality	Inline <code>onclick=</code> , <code>eval()</code> / <code>document.write()</code> , deprecated tags, inline scripts over 50 KB, jQuery < 3.x (known XSS CVEs)	serious–minor
Broken links auto-crawl phase only	Dead links tested with the logged-in user's cookies and real User-Agent, 8-way parallel, HEAD→GET fallback, and 401/403/429/503 are not counted as broken . Plus empty <code>src</code> slots, dead <code>#fragment</code> anchors, malformed <code>mailto:</code>	serious

SAME CHECKS, DIFFERENT AUTHORITY

In manual mode these run as a silent co-pilot: shown beside the tester's own observations, counted in the report, and **never** allowed to change the verdict. Broken links wait for the auto-crawl phase because they fire up to 30 HTTP requests and would stall the live tracker.

What only automation can see — and what only a human can

Automated-only category	Why a manual session cannot run it
Security headers	Needs real response headers. A manual session has none, so the check would report <i>every</i> header missing on <i>every</i> page — a pure false positive.
Mobile	The tester's window is a desktop viewport, where every ordinary control looks like an undersized 44 px tap target.
Performance	Needs <code>page_load_ms</code> measured by the crawler. Timing sampled mid-manual-browsing is meaningless.
API issues	Needs the <code>request</code> / <code>response</code> listener that records <code>elapsed_ms</code> , status and content-type. Manual mode does not attach it.
Form validation	The blank-every-field <code>checkValidity()</code> probe is disruptive on a live interactive session.
Functional (steps)	Automation <i>executes</i> your written steps. In manual mode a human ticks the same steps instead.
Dead controls	Opt-in per test case: clicking controls on a live site can submit a form or log the crawler out.

orange

Human-only — judgement

Three channels no machine has: the **step checklist** (passed / failed / skipped), the **expected-result decision** (met or not met), and **reported issues** with a category and severity the tester chooses.

blue

Automated-only — reach

The crawler reaches 100 pages in minutes and re-runs the identical audit after every hotfix. No tester could do that consistently.

Visual and UX quality, copy accuracy, and “does this feel right” sit in the second column and nowhere else. That asymmetry is the entire argument for running both modes.

Accessibility: axe-core injected into the live page

axe-core 4.10.2 is vendored as a file — no npm install, no CDN call, works offline.

```
# backend/modules/runner/routes.py : 642
AXE_PATH = os.path.join(os.path.dirname(__file__), 'axe.min.js')
# image-alt is owned by check_missing_alt already -- running both would
# charge the score twice for one defect.
AXE_RULES_COVERED_ELSEWHERE = {'image-alt'}

def check_accessibility_axe(page):
    if not os.path.exists(AXE_PATH):
        return None
    try:
        page.add_script_tag(path=AXE_PATH) # inject axe into the page
        # axe.run() returns a Promise; Playwright auto-awaits it
        results = page.evaluate(
            """() => axe.run(document, {
                runOnly: { type: 'tag',
                    values: ['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'] },
                resultTypes: ['violations']
            }).then(r => r.violations)"""
        )
    except Exception as e:
        logger.warning("axe-core failed, using heuristic ally: %s", e)
        return None # caller falls back to heuristics
```

- **Tags run:** `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa` — WCAG 2.1 A and AA criteria, `violations` only.
- **Severity and weight:** axe's own `impact` becomes the runner severity, and the count of failing elements feeds a logarithmic weight.
- **Fallback:** if the script fails to load, a heuristic check runs — unlabelled buttons and inputs. Placeholder text is deliberately *not* accepted as a label.
- **Scoring integrity:** if axe never ran, the category is left out of the weighted average entirely. A check that never ran is not a check that passed.

Judgement inside a check: the security header logic

This is what stops a security check from being a naive header-presence linter.

```
# backend/modules/runner/routes.py : 600
def check_security(page_url, response_headers):
    findings = []
    is_https = page_url.startswith('https://')
    if not is_https:
        findings.append({
            'issue': 'Site is using insecure HTTP instead of HTTPS',
            'severity': 'critical', 'header': 'protocol',
        })
    headers_lower = {k.lower(): v for k, v in response_headers.items()}
    csp = (headers_lower.get('content-security-policy') or '').lower()
    missing = []
    if 'content-security-policy' not in headers_lower:
        missing.append({'header': 'Content-Security-Policy',
            'purpose': 'Protects against XSS'})
    # CSP frame-ancestors is the modern replacement for
    # X-Frame-Options. A site using it IS protected and must not
    # be told otherwise.
    if 'x-frame-options' not in headers_lower \
        and 'frame-ancestors' not in csp:
        missing.append({'header': 'X-Frame-Options',
            'purpose': 'Prevents clickjacking'})
    # HSTS only means anything over HTTPS. On an HTTP page the
    # protocol is already reported above, and charging twice for
    # one root cause drags the score down for a single fault.
    if is_https and 'strict-transport-security' not in headers_lower:
        missing.append({'header': 'HSTS', 'purpose': 'Forces HTTPS'})
    return findings
```

- **No false positive on modern CSP** — a site using `frame-ancestors` is not told it is missing X-Frame-Options.
- **No double-charging** — HTTP-and-no-HSTS is one root cause, not two penalties.
- **Escalation** — plain HTTP is the only security finding rated critical; the aggregated header gap is serious.

Written steps become real browser actions

Plain English, one action per line, 15 supported actions. This catches functional failures a pure crawler never could.

```
# backend/modules/runner/steps.py : 13
Open https://shop.example.com
Type alice@example.com into Email
Type hunter2 into Password
Click Sign in
Expect text Welcome back
Expect url contains /dashboard

Select Extra Large in Size
Tick the Terms checkbox
Expect no "Item removed"      (quoting keeps multi-word values intact)

def parse_step(line):
    raw = (line or '').strip()
    stripped = _NUMBERING.sub('', raw).strip() # drop "1." / "2)"
    for action, pattern in _PATTERNS:        # first match wins
        match = pattern.match(stripped)
        if not match:
            continue
        g = match.groupdict()
        return {'raw': raw, 'action': action,
                'target': _clean(g.get('target')),
                'value': _clean(g.get('value'))}
    # A bad line is data, not a crash -- and it is NOT a passing step.
    return {'raw': raw, 'action': 'unknown', 'target': '', 'value': ''}
```

blue

Stop at first failure

Later steps are recorded as *skipped*, not re-reported as new defects — downstream failures are consequences, not bugs.

ok

Unreadable is not failing

A step the parser cannot read is **not** charged against the site. It never reached the application, so it surfaces in the coverage notes instead.

warn

Smart locator resolution

Targets resolve by role, then label, then placeholder, then `name`, then text.

Navigating a modern SPA without losing the session

The crawl clicks links instead of reloading, so sessionStorage and in-memory auth state survive every route.

```
# backend/modules/runner/routes.py : 2993
if page_idx == 0:
    # First navigation: full HTTP load so we can capture security
    # headers and verify the page is reachable.
    response = page.goto(page_url, timeout=30000,
                        wait_until='domcontentloaded')

    page_status = response.status
    response_headers = dict(response.headers)
    saved_first_headers = dict(response.headers)
else:
    # SPA-internal click navigation: keeps React mounted and auth
    # state alive. Falls back to page.goto inside navigate_spa if
    # no matching link exists on the current page.
    _, response = navigate_spa(page, page_url)
    if response is not None:
        page_status = response.status
        response_headers = dict(response.headers)
    else:
        page_status = 200 # click nav succeeded
        response_headers = saved_first_headers # CSP/HSTS unchanged
page_load_ms = int((time.time() - nav_start) * 1000)

# Wait for the DOM to stop changing rather than guessing a duration.
page.wait_for_load_state('networkidle', timeout=5000)
wait_for_page_settled(page) # polls href + node count every 50ms
```

- `navigate_spa` finds a matching `<a>` and clicks it — trailing-slash insensitive, so `/about` and `/about/` both stay on the click path.
- `wait_for_page_settled` polls `location.href + '|' + querySelectorAll('*').length` until it is steady for a quiet window. Fixed sleeps were the old approach and they were wrong.

The health score: why it is not a naive count

```
SEVERITY_PENALTY = critical 10, serious 5, moderate 2, minor 1. SCORE_SCALE = 20.

# backend/modules/runner/routes.py : 1303
def finding_weight(finding):
    """axe reports one finding per rule together with the number
    of elements breaking it, so a rule broken by 200 elements has
    to cost more than one broken by a single element."""
    count = 1
    if isinstance(finding, dict):
        raw = finding.get('count')
        if isinstance(raw, int) and raw > 1:
            count = raw
    return 1 + math.log10(count)

def category_subscore(category, items):
    """Score out of 100 with diminishing returns:
    100 * SCALE / (penalty + SCALE).
    The old score was 100 - count*weight, which hit zero on
    any real site and made a page with 20 problems look
    identical to one with 200."""
    penalty = category_penalty(category, items)
    return int(round(100 * SCORE_SCALE / (penalty + SCORE_SCALE)))
```

15 categories, weights summing to 100: functional 25, security 16, accessibility 14, broken links 13, console 11, API 10, validation 10, performance 9, execution 8, URL 8, UI 7, code 7, mobile 6, SEO 4, missing-alt 2.

```
# backend/modules/runner/routes.py : 1393
def calculate_health_score(findings_by_category, measured=None):
    """Weighted average of the per-category subscores, 1..100.

    `measured` names the categories this run actually checked.
    Anything outside it is left out of the average rather than
    counted as perfect, because a check that never ran is not a
    check that passed."""
    total_weight = weighted_sum = 0
    for category, weight in CATEGORY_WEIGHT.items():
        if measured is not None and category not in measured:
            continue
        items = findings_by_category.get(category) or []
        weighted_sum += weight * category_subscore(category, items)
        total_weight += weight
    if not total_weight:
        return 100
    return max(1, min(100, int(round(weighted_sum / total_weight))))
```

THE PROBLEM THIS FIXED

The old linear formula saturated at zero. Two sites with 20 issues and 200 issues both scored 0 — which destroyed the ability to compare two runs, the only thing a trend chart is for.

De-duplication: the same footer bug is not 40 bugs

De-dup keys are per-category: broken link by URL, security by header, accessibility by rule (not element selector), and never by screenshot or timing.

```
# backend/modules/runner/routes.py : 2156
def merge_findings(all_pages_findings):
    """Combine per-page findings into one summary, de-duplicating
    issues that repeat across pages. The SAME footer broken-link,
    missing security header, or ally rule appears on every page of
    a site; counting it once per page inflated the issue total and
    wrongly tanked the health score. Per-page detail is still
    preserved in the crawled_pages records -- this only affects
    the summary."""
    merged = {category: [] for category in FINDING_KEYERS}
    seen = {cat: set() for cat in merged}
    for page_findings in all_pages_findings:
        for category, items in page_findings.items():
            if category not in merged:
                continue
            for item in items:
                key = finding_key(category, item)
                if key is not None:
                    if key in seen[category]:
                        continue
                    seen[category].add(key)
                    merged[category].append(item)
    return merged
```

orange

The bug

A single dead `/dead` link in the footer was reported **once per page**. 8 missing-header findings became 2. Issue counts were inflated and the health score was wrong.

ok

The fix

Per-category de-duplication keys. The key must never include volatile data — screenshot path, timing, page order — or nothing would ever merge.

blue

The verification

An end-to-end crawl against a controlled site with deliberate issues: 4 pages crawled, `/dead` collapsed to **1**, security collapsed to **2**, and **no false positives** on the valid pages.

05

Technology Deep Dive

What each piece of technology actually does in the testing pipeline, and why it was chosen.

- Playwright as the single automation engine
- axe-core vendored offline for WCAG auditing
- ThreadPoolExecutor for job isolation
- MySQL as the single source of truth

What plays which role

Technology	Role in manual testing	Role in automated testing
Playwright (Python, sync API)	Launches and manages the headed Chrome window; screenshots every navigation	Launches headless Chromium; executes steps; clicks links; reads DOM, console and network
axe-core 4.10.2 (vendored JS)	Runs on each page the tester visits, as supporting evidence	Injected via <code>add_script_tag</code> ; WCAG 2.1 A/AA <code>violations</code> only
Threading (<code>ThreadPoolExecutor</code> , 2 workers)	One dedicated thread owns the manual session; Flask threads only signal it	JobManager worker per run; 8-way pool inside the broken-link check
Pillow	—	Crops each full-page screenshot around a finding's bounding box (25 px padding)
requests	—	Parallel link validation, forwarding the browser's cookies and real User-Agent
MySQL (9 tables)	Run row, step evidence, tester issues, verdict	Run row, per-page findings, score breakdown, coverage, regression
Flask-JWT-Extended	Authenticates every manual endpoint	Authenticates the submit and the poll; ownership checked on both
ReportLab	—	Server-side PDF export of the report bundle
Gemini (REST)	Can explain a tester-reported issue	Suggests test cases, analyses machine findings, proposes fixes

What each mode needs — and why no API key is involved

Neither manual nor automated testing uses an API key. Issue detection makes no call to any third-party API: both modes run entirely locally against the target site, and both work offline.

Mode	What it actually requires
Automated	Playwright's locally installed Chromium, MySQL credentials, JWT secret. Nothing else.
Manual	The user's installed Google Chrome (falls back to bundled Chromium), MySQL credentials, JWT secret. Nothing else.

```
Reached only from modules/ai/. Grepping the runner and testcases modules for AI references returns zero hits.

# The ONLY outbound call in the entire project.
# backend/modules/ai/service.py : 40
requests.post(
    f'https://generativelanguage.googleapis.com/v1beta'
    f'/models/{_get_model()}:generateContent',
    params={'key': api_key},          # ← the one and only key
    json=payload, timeout=60)
```

The project has exactly one key

GEMINI_API_KEY, loaded from backend/.env and consumed at one line. It serves four advisory endpoints — analyze-bug, suggest-tests, suggest-fix and chat — every one of them on-demand, none of them in the run path. GET /api/ai/health is the only endpoint that reads the key without calling out; it just reports whether one is configured.

So the failure modes are separate

If the key is invalid, rate-limited or the credits are exhausted, automated runs and manual runs are entirely unaffected — they keep working. Only the AI page degrades, mapping to 429 for rate limits and 503 for exhausted credits.

WHY THIS MATTERS MORE THAN IT LOOKS

That separation is deliberate, and it is why the results are trustworthy. A model cannot hallucinate a passing grade, because no model is anywhere near the scoring path. The health score, the Pass/Fail verdict and bug creation are all deterministic code — the AI only explains and suggests.

Authenticated testing: capture once, replay on every run

Three files per project: `<id>.json`, `<id>.session_storage.json`, and `<id>.landing`.

```
# backend/modules/projects/session_capture.py : 125
os.makedirs(SESSIONS_DIR, exist_ok=True)
path = os.path.join(SESSIONS_DIR, f'{self.project_id}.json')
context.storage_state(path=path) # cookies + localStorage

# Capture sessionStorage too. Playwright's storage_state only
# saves cookies + localStorage; many SPAs keep their auth flag
# in sessionStorage, so without this the crawler appears
# anonymous to the SPA.
session_storage = page.evaluate(
    "() => { const o={}; for (let i=0;i<sessionStorage.length;i++)"
    " { const k=sessionStorage.key(i);"
    "   o[k]=sessionStorage.getItem(k); } return o; }"
) or {}
```

Replayed on every page load, because Playwright's storage_state cannot carry it.

```
# backend/modules/runner/routes.py : 2798
if session_used:
    ss_data = _session_storage_dict(tc.get('project_id'))
    if ss_data:
        payload = json.dumps(ss_data)
        context.add_init_script(
            f`() => {{ try {{ const d = {payload}; "
            "for (const k in d) "
            "  sessionStorage.setItem(k, d[k]); "
            "} catch(e) {{ }}})();"
        )
```

1

Open & Login — a real window opens

2

Human signs in: SSO, OTP, captcha, OAuth

3

Click **I'm logged in** → 3 files saved

4

Every later automated run replays them

5

Crawl starts at the **landing URL**, not `/`

HONEST FAILURE

Sessions are checked for expiry before every run (`_session_expiry_info` scans for `token` | `session` | `auth` | `jwt` | `sid` cookies). An expired session falls back to anonymous crawling *and* surfaces a warning — it never silently crawls the public site and calls it a pass.

From a finding to a tracked bug

1

Finding raised by a check
or by the tester

2

Result modal shows it with
cropped screenshot evidence

3

Auto-Create Bugs
one button, whole batch

4

Per-category repro steps
auto-written

5

Bug row created, linked to
the run and test case

A manual run stores its bookkeeping as a `<i>string</i>` under 'note'. Without this allowlist, naive iteration would have created one bug per character.

```
# backend/modules/bugs/routes.py : 732
# Manual runs keep their record under these keys: metadata, the
# pages visited, the whole checklist (passed steps included) and
# the run summary. None of that is a defect.
MANUAL_BOOKKEEPING_KEYS = {
    'manual', 'note', 'pages', 'steps', 'summary',
    'expected_result', 'expected_met',
}

# Whitelist of genuine runner categories -- anything else is
# skipped rather than guessed at.
RUN_FINDING_CATEGORIES = {
    'broken_links', 'console_errors', 'security_issues',
    'accessibility_issues', 'performance_issues',
    'api_issues', 'validation_issues', 'seo_issues',
    'ui_issues', 'code_issues', 'url_issues',
    'mobile_issues', 'missing_alt_images', 'functional_issues',
}
```

- **Reproduction steps are written per category** — a broken link becomes “open the page → find the link → click it” expecting HTTP 200.
- **Severity respects the finding's own grade** before falling back to the category default, so a critical failure cannot be downgraded to Minor just because its category has no entry.
- **Idempotent per test case** and per-finding failures are caught, so one malformed finding cannot abort the batch.

Reports, dashboards and the AI layer

Reporting — 9 endpoints

- Health-score trend over time, per project
- Pass rate, recent runs, run statistics with filters
- Bug breakdown by status and severity
- Top issue categories across the whole platform
- Detected issues, filterable by **MANUAL** / **AUTOMATED**
- **PDF export** — ReportLab builds the document in memory and streams it back as `qa-platform-report-YYYY-MM-DD.pdf`

BOUNDARY

The AI never decides a verdict. It analyses, suggests and explains. Scoring, verdicts and bug creation are all deterministic code — which is what makes the reports trustworthy enough to act on.

AI — Google Gemini

- **Bug analysis** → explanation, why it matters, how to fix, and a code example
- **Test suggestions** → 8 cases across functional, security, performance, accessibility, mobile and SEO — each labelled automated or manual depending on whether a script could judge it
- **Fix suggestions** → what it is, user impact, the fix, an example
- **Chat** → with live platform context: project and test-case counts, the last 5 bugs
- Every prompt demands strict JSON, and every endpoint degrades gracefully when parsing fails
- Rate limits map to **429**, exhausted credits to **503**

06

Quality of the Platform

A QA tool that is not itself tested is a liability. We built a real test suite for it — and it passes.

- 312 backend tests, all passing
- 14 frontend tests, all passing
- ESLint clean, production build green
- Live end-to-end crawls against real Chromium

The backend suite — 312 tests across 18 files

Test file	Tests	What it protects
test_detection_quality.py	78	Every false-positive guard: status classification, de-duplication, mobile skipping, noise filtering
test_steps.py	41	The step DSL — all 15 actions, parsing order, locator resolution
test_runner_helpers.py	32	Scoring, coverage roll-up, URL normalisation, crawl patterns
test_manual_workspace.py	22	Manual session lifecycle, checklist state, verdict summarisation
test_extended_checks.py	21	URL health, API quality, page resources, UI consistency, code quality
test_runner_logic_fixes.py	17	Regression tests for each bug we actually fixed
test_run_separation.py	12	The manual / automated evidence contract
test_admin_access.py	11	Role checks, deactivated-account blocking, admin-only routes
test_crawl_queue.py	10	Frontier seeding, login-page pruning, pattern caps
test_testcase_workflow.py	8	Create, update, delete, ownership
test_page_settle.py	7	DOM-settle detection
test_page_checks_e2e.py	6	Real Chromium: every category returns, evidence crops, no invented findings
test_steps_e2e.py	6	Real login workflow, wrong-password failure, screenshot per step
test_app.py	5	App factory, blueprint registration, error handlers
test_live_run_e2e.py	3	Full live automated run and full manual workflow, headless
test_jobs.py	3	Job submit, poll, cancel, eviction
test_manual_run_e2e.py	3	Session capture, clean finish, auto-trigger timing
test_session_capture.py	3	Storage-state and sessionStorage capture

Executed results, not claims

312

Backend tests passing

pytest, real headless Chromium, 69s

14

Frontend tests passing

Vitest + Testing Library, jsdom

0

Lint errors

ESLint 10 across the whole frontend

0

Build errors

Vite 8 production bundle, 4.5s

Reproduce exactly what was run for this deck.

```
# Terminal 1 -- backend
cd backend
pytest
# 312 passed, 41 warnings in 69.01s (0:01:09)

# Terminal 2 -- frontend
cd frontend
npm test      # Test Files 2 passed (2)
              # Tests 14 passed (14)
npm run lint  # eslint . → exit 0
npm run build # built in 4.51s → exit 0
```

- **Not mocked** — the end-to-end tests launch real Chromium and audit a controlled local site with deliberate defects planted in it.
- **Not just unit** — `test_live_run_e2e.py` runs the actual `_perform_run` with the database stubbed, then asserts on the result.
- **The verification that mattered** — 11 of 11 assertions passed on that crawl: 4 pages, a real `duration_ms`, the `/dead` link collapsed to 1, security collapsed to 2, and no false positives on the valid pages.
- **Frontend** — 14 component tests cover the report rendering and the run modal, including the NaN-safety guards.

07

Engineering Challenges

The bugs we found by running the tool on real sites — and the reasoning behind each fix.

- Four production bugs diagnosed with data, not guesswork
- Every fix verified by executing it, never by claiming it
- A logging crash traced to two processes and one locked file

Four real bugs, diagnosed with evidence

Symptom	Actual root cause	Fix
<code>--- Logging error ---</code> flooding the terminal during crawls	Two causes. (1) <code>RotatingFileHandler</code> failing on Windows (<code>WinError 32</code> , file locked) — the Flask debug reloader runs two processes , both holding <code>app.log</code> open. (2) The console handler's <code>cp1252</code> encoding choking on non-ASCII.	Force console to UTF-8 with <code>errors='replace'</code> ; a <code>_SafeRotatingFileHandler</code> that swallows rotation failures; attach the file handler only to the serving process via <code>WERKZEUG_RUN_MAIN</code>
Every link behind the login reported as broken	<code>check_broken_links</code> used bare <code>requests.head()</code> with no browser cookies , so every login-gated link returned 302/401/403. Many servers also reject <code>HEAD</code> outright.	Forward the browser's cookies and real User-Agent into a per-thread session so links are tested <i>as the logged-in user</i> ; fall back to <code>GET</code> when <code>HEAD</code> is rejected; stop treating 401/403/429/503 as broken
Mobile findings that were pure noise on authenticated crawls	Authenticated crawls deliberately run a desktop viewport to keep the session valid — but the 44 px tap-target and horizontal-scroll checks still ran.	Skip the mobile check whenever a session is in use: <code>is_mobile = not session_used</code>
Issue counts inflated, health score wrong	<code>merge_findings</code> had no de-duplication , so one footer link or one missing header was counted once per page — 8 header findings instead of 2.	Per-category de-duplication keys: broken link by URL, security by header, accessibility by rule

The hard one: “Job not found” and NaNs

This is the honest state of the project: the user-facing symptom is fixed and explained, the durability gap is documented, not hidden.

```
# Symptom: an automated crawl of Amazon died partway
# (page 41 of 75), the result modal showed "Job not found",
# and every duration field rendered as "NaNs".

# Chain of causes, found by reading app.log:

# 1. The async job registry is IN-MEMORY (jobs.py). Any backend
#    restart destroys every in-flight and finished job.
#
# 2. The Flask debug reloader restarts the process on every .py
#    save. app.log showed ~130 restarts during the session --
#    several triggered by our own edits while the server ran.
#
# 3. The frontend polled, got a 404, built an error object with
#    no duration_ms, and then evaluated:
#    (undefined / 1000).toFixed(1) → "NaNs"

# Fixes:
# * fmtSecs() returns an em-dash instead of NaN when timing is
#   missing -- a missing value is displayed, never invented.
# * A 404 mid-poll now says: "the run was interrupted -- the
#   backend restarted", instead of "Job not found".
# * Operational: run the backend with the reloader disabled
#   for real crawls (use_reloader=False).
# * Planned: persist job state to a MySQL run_jobs table, so
#   polls survive a restart at all.
```

TAKEAWAY

The general lesson: **an error message that lies is worse than a crash.** “Job not found” sent us looking for a job-lookup bug. The truthful message — “the backend restarted” — identified the cause immediately.

08

Security & Safeguards

A testing tool takes a target URL, stores credentials and writes files. That is a large attack surface, so it is defended deliberately.

- SSRF protection with DNS resolution and private-IP blocking
- JWT auth plus ownership checks on every object
- Browser sessions and evidence treated as sensitive data

Safeguards built into the test engine

red

SSRF protection

Every test URL is DNS-resolved and the resolved IP is rejected if it is private, loopback, link-local, multicast, reserved or unspecified — which also blocks `169.254.169.254` cloud metadata. DNS failure fails *open*; a successful resolve to a private IP fails *closed*. Escape hatches exist for local dev.

blue

Auth on everything

Every runner, project, bug and report endpoint sits behind `@jwt_required()`, and every object read or write is ownership-checked against the owning project, not just the user id.

ok

Deactivated accounts

Tokens do not expire in this configuration, so a blocklist loader re-checks `users.is_active` on every single request.

warn

Rate limiting

Flask-Limiter is applied per endpoint rather than globally, so login and AI endpoints are protected without throttling a 100-page crawl.

purple

Anti-automation detection

`navigator.webdriver` is masked, the manual runner prefers the user's real installed Chrome, and `--disable-blink-features=AutomationControlled` is set.

red

Sensitive data

Saved sessions contain live cookies and tokens. They are excluded from version control, stored outside the web root, and treated as private runtime data.

09

Limitations & Roadmap

What is not solved yet, and what is next — ordered by impact per unit of effort.

- Job durability: in-memory registry loses runs on restart
- Authenticated crawl unverified end-to-end against a live SPA
- Crawl frontier has no URL-pattern de-duplication

LIMITATIONS

Honest gaps, and what we would do next

#	Limitation	Impact	Planned fix
1	Job durability. The job registry is in memory, so any backend restart loses in-flight and finished runs. A 75-page crawl (~40 min) is fragile to any restart.	High — long runs can be lost entirely	P0. Persist job state to a MySQL <code>run_jobs</code> table so polls survive restarts. A Celery/RQ + Redis worker is the heavier alternative. Interim: run with the reloader off
2	Authenticated crawl, unproven live. Every building block is individually verified (cookie forwarding, session-preserving navigation, sessionStorage replay, landing-URL warmup), but a full real login → automated crawl against a live SPA has not been run.	High — the flagship path is untested end to end	P1. Capture a real session on a live authenticated app, run the crawl, and confirm <code>session_used=True</code> , sessionStorage injected, landing page past <code>/login</code> , and protected links <i>not</i> flagged broken
3	Crawl frontier control. A large site queued 380+ URLs for a 75-page cap, many near-duplicate product pages.	Medium — wasted wall-clock, less meaningful <code>max_pages</code>	P2. Collapse ID-like URL segments into patterns (already implemented for capping), add per-section caps, and optionally test pages in parallel
4	Manual auto-trigger. Can still fire early if a project's start URL is not recognised as a login page. Button-only mode was offered, not applied.	Low — a one-line change away	P4. Remove the timer, keep the explicit "Auto-crawl from here" button
5	Health-score weights are untuned now that duplicates are gone, so scores will rise across the board.	Low	P3. Re-tune the 15 category weights against real post-fix data
6	OTP module unwired. The endpoints and the wait loop exist and are tested, but nothing calls them — OTP happens inside the human's own browser window.	Low — scaffolding, not a broken path	Wire it into the manual flow, or delete it. Unused code is a liability

What this project taught me

- **A check that cries wolf is worse than no check.** Every false-positive guard in the 78-test `test_detection_quality.py` exists because an early version told users their site was broken when it was not.
- **Fix the root cause, not the symptom.** `NaNs` was a formatting bug; the real problem was an in-memory job registry. Patching only the formatter would have hidden a lost 40-minute run.
- **De-duplicate before you score.** Counting one footer bug once per page made every score meaningless. Fixing aggregation was worth more than adding five new checks.
- **Run your tool on something real, early.** All four production bugs appeared only against a live 75-page site, never in a unit test.
- **Non-measurement is not success.** A category that never got checked is excluded from the score instead of counted as perfect. This applies to test coverage too.
- **Verify by executing.** Every fix in this project was validated with a real harness and a recorded result — never shipped on a claim.

The one-paragraph summary

A testing platform earns trust the same way the tests it runs do: by being honest about what it checked, honest about what it found, and honest about what it could not check at all. That is why the manual verdict is protected from automation, why unmeasured categories are excluded from the score, and why a broken run is reported as interrupted rather than quietly passing.

IF YOU REMEMBER ONE LINE

Manual testing supplies judgement. Automation supplies coverage. The engineering is in keeping them honest with each other.

THANK YOU

Questions?

Manual testing

Headed real Chrome driven by a backend worker.
Written steps become a live checklist, every navigation is screenshotted, and the tester's verdict is protected from being overridden by the automatic checks running alongside it.

Automated testing

Headless Chromium via Playwright, 16 checks per page, BFS crawl up to 100 pages, authenticated by replaying a saved session, de-duplicated findings and a weighted health score.

The proof

312 backend tests and 14 frontend tests passing, ESLint clean, production build green — with live end-to-end crawls against real Chromium.